CAMPUS PLACEMENT TECHNICAL CASE STUDY

FLASH-SALE CHECKOUT SYSTEM DESIGN

Designing a High-Concurrency Reservation System for Limited Inventory

Prepared by **Srri Hari T R**

Role: Full-Stack Software Engineer

GitHub Repository: https://github.com/SrriHariTR/goML-flash-sale

1. Executive Summary

This report presents the architecture and working implementation of a high-concurrency flash-sale checkout system designed to handle a burst of 100,000 concurrent requests competing for a strict inventory of 500 smartphones. To eliminate the legacy platform's overselling bug (540 checkouts for 500 units), database row-lock timeouts, and UI freezing, the system implements a decoupled, multi-layered defense using **React + TypeScript**, **Nginx**, **Node.js (Express)**, **Redis**, **Apache Kafka**, and **PostgreSQL**.

Client-side button throttling, Edge CDN static asset delivery (< 200KB bundle, < 1.5s Time-to-Interactive), and idempotency keys (Idempotency-Key headers) prevent duplicate payload spamming at the UI layer. At the middleware layer, Nginx and Redis absorb massive traffic spikes via atomic rate-limiting and single-threaded in-memory inventory gatekeeping, while **HTTP 202 Accepted** responses decouple checkout requests from heavy persistence operations. Ultimate data integrity is guaranteed through atomic conditional SQL updates in PostgreSQL paired with the **Transactional Outbox pattern** and **Kafka background workers** for asynchronous event processing and automated reservation expiry cleanup.

Local container validation across 510 concurrent batch requests (following 10 initial rate-limit test reservations) yielded **490 accepted reservations (HTTP 202)** and **20 stock-exhausted conflict rejections (HTTP 409)** in 3.3 seconds, terminating at exactly **500 reserved units and 0 oversold items**.

2. Problem Statement and Requirements Analysis

2.1 Problem Statement

An e-commerce platform is planning a flash sale for 500 smartphones at exactly 8:00 PM. Based on expected demand, more than 100,000 concurrent customer requests may reach the checkout endpoint within the first 10 seconds.

The main challenge is to process a large number of requests while maintaining inventory accuracy, preventing duplicate reservations, and keeping the system responsive. If multiple requests attempt to

reserve the same remaining units simultaneously, an incorrectly designed system could oversell the available inventory.

The system must therefore coordinate concurrent requests, reject reservations when stock is exhausted, and handle failures without losing reservation events or creating duplicate operations.

2.2 Functional Requirements

1. Display the flash-sale product and its reservation status.
2. Accept customer reservation requests through an API.
3. Ensure that no more than 500 units are reserved.
4. Prevent duplicate reservations using idempotency.
5. Reject requests when inventory is exhausted.
6. Apply rate limiting to reduce excessive requests.
7. Process reservation events asynchronously.
8. Track reservation status and handle reservation expiry.
9. Provide a health endpoint to check service and dependency availability.

2.3 Non-Functional Requirements

- **Scalability:** Support a large traffic spike through horizontal scaling and asynchronous processing.
- **Consistency:** Maintain accurate inventory under concurrent requests.
- **Reliability:** Handle transient failures and avoid losing committed reservation events.
- **Performance:** Keep request processing efficient and measure latency under load.
- **Security:** Validate inputs, protect secrets, and use authentication and HTTPS in production.
- **Observability:** Record useful logs, health status, errors, and performance metrics.

2.4 Key Constraints

The available inventory is fixed at 500 smartphones. Inventory correctness takes priority over accepting every request. Requests arriving after stock is exhausted must be rejected safely.

The target of 100,000 concurrent users is a design requirement, not a verified performance result. The implementation must be subjected to a dedicated load test before claiming that it meets this target.

3. Proposed System Architecture

3.1 Architecture Overview

The flash-sale system follows a layered architecture that separates the user interface, request handling, inventory management, and asynchronous processing.

The React and TypeScript frontend allows customers to submit reservation requests and view reservation details. Nginx serves the frontend and forwards API requests to the Express backend. The backend coordinates reservation operations, using PostgreSQL as the authoritative source for inventory and Redis for rate limiting.

Kafka handles asynchronous reservation events, while a background worker processes those events and supports reservation-related tasks. A transactional outbox helps ensure that events associated with committed database transactions can be published reliably.

This separation improves maintainability and provides a foundation for scaling individual components as demand increases.

3.2 Components and Responsibilities

1. **React + TypeScript:** Displays the product, accepts reservation requests, and shows reservation results.
2. **Nginx:** Serves the frontend and routes API requests to the backend.
3. **Express API:** Validates incoming requests, applies reservation logic, and returns appropriate responses.
4. **PostgreSQL:** Stores inventory and reservation records and enforces inventory limits through atomic database operations.
5. **Redis:** Supports rate limiting to control excessive incoming requests.
6. **Kafka:** Carries reservation events for asynchronous processing.
7. **Background Worker:** Consumes events and performs background reservation-related processing.
8. **Transactional Outbox:** Stores events within the same database transaction as the associated state change, allowing a publisher to forward them to Kafka.

3.3 Request Flow

A customer submits a reservation request through the frontend. Nginx forwards the API request to the Express backend. The backend checks the request and applies rate limiting before attempting to reserve inventory in PostgreSQL.

If inventory is available, the database transaction reserves a unit and records the related event in the outbox. The event is subsequently published to Kafka and processed by the background worker. If stock is unavailable, the API rejects the reservation rather than allowing inventory to exceed the limit.

The database remains the source of truth for inventory. Redis and Kafka support traffic control and asynchronous processing; neither replaces the database's inventory guarantees.

3.4 Frontend Strategy & Client Optimization To meet the strict **< 200KB bundle size** and **< 1.5s Time-to-Interactive (TTI)** targets while preventing client-side spamming, the React + TypeScript frontend implements four core optimizations:

- **UI Throttling & Debouncing (Instant Button Lock):** Upon the first click of the *"Reserve your phone"* button, the local UI state immediately locks (isSubmitting = true), disabling the button and attaching a client-generated Idempotency-Key before dispatching the network request. This eliminates repetitive API payloads from rapid user clicks.
- **Edge CDN & Asset Pre-fetching:** All static assets (HTML, CSS, minified JS bundles, and compressed WebP product imagery) are served via an Edge Content Delivery Network (CDN) ahead of Nginx. Tree-shaking and code-splitting keep the client bundle under **200KB**, ensuring a **TTI < 1.5s** and freeing the backend origin servers exclusively for dynamic checkout API calls.
- **Optimistic UI Updates:** The interface immediately transitions into a smooth *"Reservation in Progress"* queue state upon click without waiting for downstream database persistence, preventing browser freezing during peak load.
- **Dynamic Polling / SSE vs. WebSockets:** Opening 100,000 concurrent stateful bidirectional WebSocket connections during an 8:00 PM spike would exhaust server file descriptors and memory. Instead, once the API returns HTTP 202 Accepted with a reservation tracking ID, the client uses lightweight **exponential-backoff HTTP polling (or unidirectional Server-Sent Events)** against cached status endpoints to render real-time reservation outcomes gracefully.

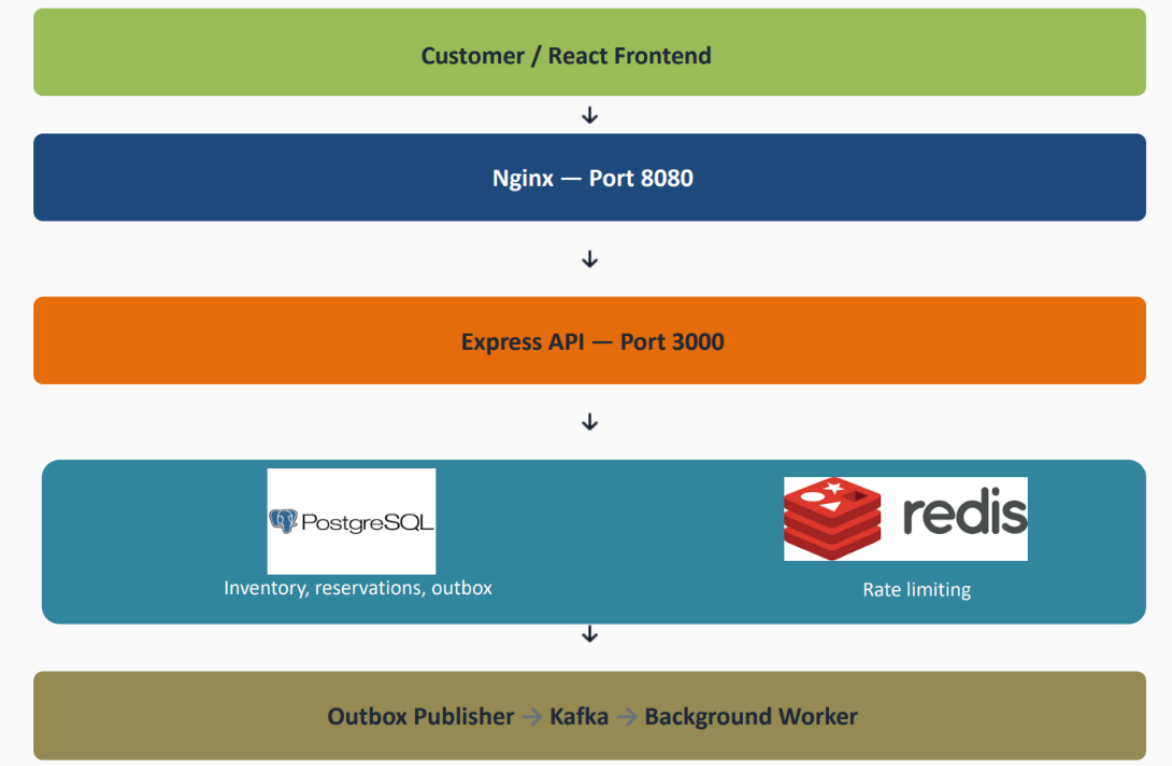


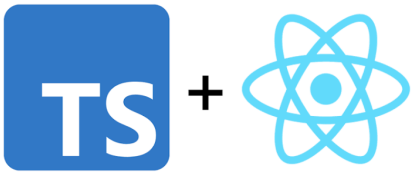
Figure 1: Architecture of the Flash-Sale Reservation System.

4. Technology Stack and Design Decisions

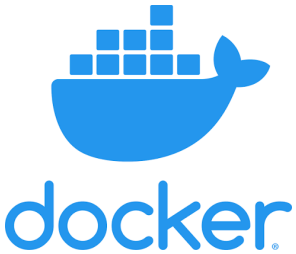
4.1 Technology Stack

The system uses the following technologies to separate the user interface, API processing, inventory management, and asynchronous event handling.

Technology	Purpose
React + TypeScript	Builds the customer-facing reservation interface
Nginx	Serves the frontend and proxies API requests
Node.js + Express + TypeScript	Implements API endpoints and reservation logic
PostgreSQL	Stores inventory, reservations, and transactional outbox records
Redis	Supports request rate limiting
Apache Kafka	Transfers reservation events for asynchronous processing
Background Worker	Consumes Kafka events and handles background tasks
Docker Compose	Runs the application and its dependencies locally

















4.2 Design Decisions

PostgreSQL for inventory consistency: Inventory is stored in a relational database that supports transactions and atomic updates. This allows concurrent reservation attempts to be controlled without exceeding the available stock.

Redis for Rate Limiting & Atomic Inventory Gatekeeping: Redis serves a dual role at the middleware tier. First, it enforces per-customer rate limits (10 requests per 60 seconds) to block bots and bad actors. Second, for 100,000-user production scale, Redis acts as an in-memory **Atomic Inventory Token Gatekeeper** using single-threaded DECR / Lua scripting (if stock > 0 then decr()). Because Redis operations execute atomically in RAM, 99,500 excess requests are rejected at the cache layer in sub-millisecond time without ever hitting PostgreSQL, allowing only the 500 valid requests to proceed to asynchronous persistence and completely decoupling heavy database writes from the HTTP cycle.

Kafka for asynchronous processing: Kafka separates event processing from the immediate API response. This allows background tasks to run independently of the customer-facing request.

Transactional outbox for event reliability: Reservation changes and their corresponding outbox records are committed in one database transaction. A publisher can then forward committed events to Kafka, reducing the risk of losing events between a database update and message publication.

Docker Compose for local deployment: Docker Compose starts the API, frontend gateway, database, cache, Kafka, and worker in a consistent local environment. This makes development and testing easier to reproduce.

4.3 Rationale

The technologies were selected to address the system's main challenges: concurrent access to limited inventory, excessive request traffic, reliable event processing, and maintainable application structure. PostgreSQL protects inventory correctness, while Redis and Kafka support traffic management and asynchronous workloads.

The current implementation is a local prototype. Additional load testing, monitoring, security hardening, and deployment configuration would be required before using it for a production flash sale.

5. Request and Reservation Workflow

5.1 Request Processing

When a customer clicks the reservation button, the frontend sends a request to the backend API. Nginx forwards the request to Express, where the system validates the input and applies rate limiting.

The backend then attempts to reserve a unit using an atomic database operation. PostgreSQL determines whether inventory is still available and ensures that concurrent requests cannot reserve more units than the configured limit.

If the reservation succeeds, the API returns a successful reservation response. If inventory is exhausted, the request is rejected with an appropriate conflict response. Excessive requests may be rejected by the rate limiter.

5.2 Atomic Inventory Reservation

Inventory correctness is the primary requirement of the flash-sale system. A simple approach that reads the stock count and updates it later can allow multiple concurrent requests to observe the same available quantity.

To avoid this race condition, inventory reservation must be performed atomically within the database. The database operation checks availability and updates the inventory as one coordinated operation.

The key invariant is:

Reserved quantity + available quantity = 500, assuming the system starts with 500 units and tracks inventory in these two categories.

The reserved quantity must never exceed 500. The database is the authoritative source for enforcing this constraint.

5.3 Idempotency

Customers may retry requests because of slow networks, timeouts, or accidental repeated clicks. Without idempotency, the same logical request could create duplicate reservations.

The system uses an idempotency key to identify a reservation request. Repeated submissions using the same key should return the existing result instead of creating another reservation, provided the implementation consistently enforces this behavior.

5.4 Transactional Outbox and Kafka

After a reservation is committed, the system needs to publish an event for asynchronous processing. Publishing directly to Kafka separately from the database transaction can create a failure window: the reservation may be saved even if event publishing fails.

The transactional outbox addresses this by saving the reservation change and its outbox event in the same database transaction. A publisher subsequently sends the event to Kafka, where the background worker consumes it.

Because events may be delivered more than once, consumers should handle duplicate processing safely, for example by tracking processed event IDs. This is a reliability design goal that must be verified against the actual worker implementation.

5.5 Reservation Expiry

The expiry test snapshot shows 522 reservations marked as expired, zero reserved units, zero sold units, and all 500 units available. This indicates that all inventory was available in the database at the time of this verification. Further testing is required to verify expiry behavior under concurrent requests and failure conditions.

5.6 Response Handling

The API distinguishes between different outcomes:

- **Accepted:** The reservation request was accepted for processing.

- **Conflict:** Inventory is exhausted or the reservation cannot be created under the current state.
- **Rate limited:** The request exceeded the configured rate limit.
- **Error:** An unexpected server or dependency failure occurred.

An accepted pending reservation is not the same as a completed purchase or confirmed payment. Payment processing is outside the scope of the current prototype.

6. Core Design & Reservation Flow

6.1 Reservation Workflow

1. The customer submits a reservation request through the React frontend.
2. The Express API validates the customer ID, quantity, and idempotency key.
3. Redis enforces the limit of 10 new attempts per customer per 60 seconds.
4. PostgreSQL atomically checks and allocates inventory within a transaction.
5. The reservation and its event are recorded in the same transaction using the transactional outbox.
6. A background worker processes reservation events through Kafka.
7. The API returns the reservation result, which the frontend displays to the customer.

6.2 Preventing Overselling

PostgreSQL is the authoritative source for inventory. Inventory allocation must use an atomic conditional update or equivalent transaction-safe locking to prevent concurrent requests from reserving the same units.

For the 500 available smartphones, once all units are reserved, further requests must be rejected. Database constraints and transaction handling also protect against duplicate allocations.

6.3 Idempotency and Reliability

Each request includes an idempotency key. Repeated requests with the same key and matching details must not create additional reservations. Reusing the key with different details returns a conflict response.

The transactional outbox stores reservation changes and their events in the same database transaction. Kafka and the background worker handle subsequent asynchronous processing, reducing the risk of losing events when message publication is interrupted.

6.4 Expected Outcome

This design protects the 500-unit inventory limit, controls excessive requests, handles retries safely, and separates immediate reservation processing from background tasks. The target of 100,000

concurrent customers requires separate load testing to validate throughput and response times.

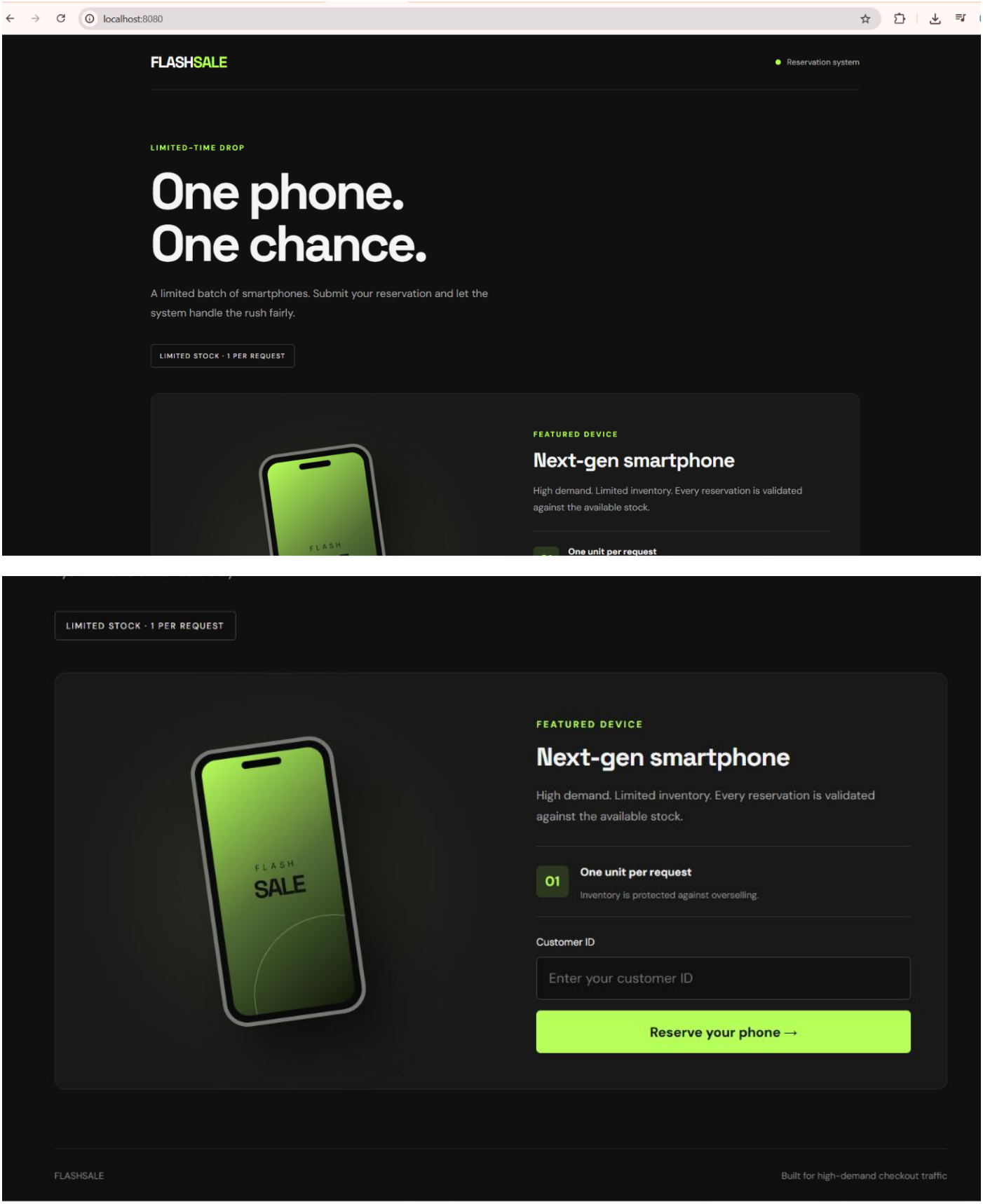


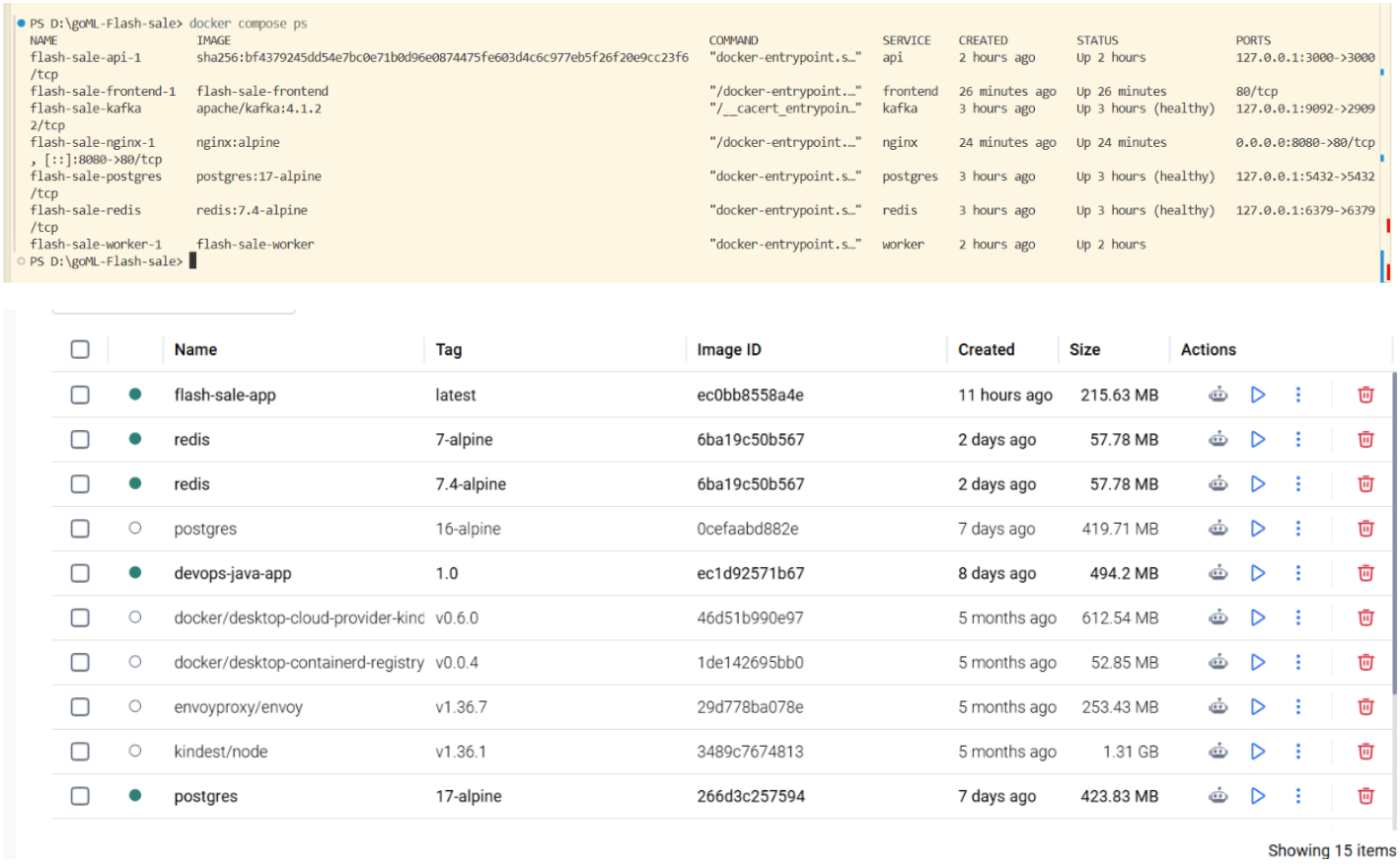
Figure 2: Flash-Sale Reservation Interface and Reservation Status

7. Implementation & Deployment

The system is implemented as a containerized application using Docker Compose, which runs the frontend, backend API, PostgreSQL, Redis, Kafka, and background worker.

- **Frontend:** React and TypeScript provide the customer reservation interface.
- **Backend API:** Express and TypeScript handle request validation, rate limiting, and reservation processing.
- **PostgreSQL:** Stores inventory, reservations, and transactional outbox events.
- **Redis:** Enforces per-customer rate limits.
- **Kafka and Worker:** Support asynchronous reservation-event processing.
- **Nginx:** Serves the frontend and routes API requests.
- **Docker Compose:** Starts and connects the application services in a consistent environment.

This architecture separates application responsibilities into independent services, simplifies local deployment, and provides a foundation for scaling individual components as demand increases.



```
PS D:\goML-Flash-sale> docker compose ps
NAME                  IMAGE                                                              COMMAND                  SERVICE    CREATED          STATUS                    PORTS
flash-sale-api-1      sha256:bf4379245dd54e7bc0e71b0d96e0874475fe603d4c6c977eb5f26f20e9cc23f6   "docker-entrypoint.s…"   api        2 hours ago      Up 2 hours                127.0.0.1:3000->3000
/tcp
flash-sale-frontend-1   flash-sale-frontend                                              "/docker-entrypoint.…"   frontend   26 minutes ago   Up 26 minutes             80/tcp
flash-sale-kafka      apache/kafka:4.1.2                                                 "/__cacert_entrypoin…"   kafka      3 hours ago      Up 3 hours (healthy)      127.0.0.1:9092->2909
2/tcp
flash-sale-nginx-1    nginx:alpine                                                       "/docker-entrypoint.…"   nginx      24 minutes ago   Up 24 minutes             0.0.0.0:8080->80/tcp
, [::]:8080->80/tcp
flash-sale-postgres   postgres:17-alpine                                                 "docker-entrypoint.s…"   postgres   3 hours ago      Up 3 hours (healthy)      127.0.0.1:5432->5432
/tcp
flash-sale-redis      redis:7.4-alpine                                                   "docker-entrypoint.s…"   redis      3 hours ago      Up 3 hours (healthy)      127.0.0.1:6379->6379
/tcp
flash-sale-worker-1   flash-sale-worker                                                  "docker-entrypoint.s…"   worker     2 hours ago      Up 2 hours
PS D:\goML-Flash-sale>
```

Name	Tag	Image ID	Created	Size	Actions
flash-sale-app	latest	ec0bb8558a4e	11 hours ago	215.63 MB	
redis	7-alpine	6ba19c50b567	2 days ago	57.78 MB	
redis	7.4-alpine	6ba19c50b567	2 days ago	57.78 MB	
postgres	16-alpine	0cefaabd882e	7 days ago	419.71 MB	
devops-java-app	1.0	ec1d92571b67	8 days ago	494.2 MB	
docker/desktop-cloud-provider-kind	v0.6.0	46d51b990e97	5 months ago	612.54 MB	
docker/desktop-containerd-registry	v0.0.4	1de142695bb0	5 months ago	52.85 MB	
envoyproxy/envoy	v1.36.7	29d778ba078e	5 months ago	253.43 MB	
kindest/node	v1.36.1	3489c7674813	5 months ago	1.31 GB	
postgres	17-alpine	266d3c257594	7 days ago	423.83 MB	

Showing 15 items

Figure 3: Docker Compose Services Used in the Implementation.

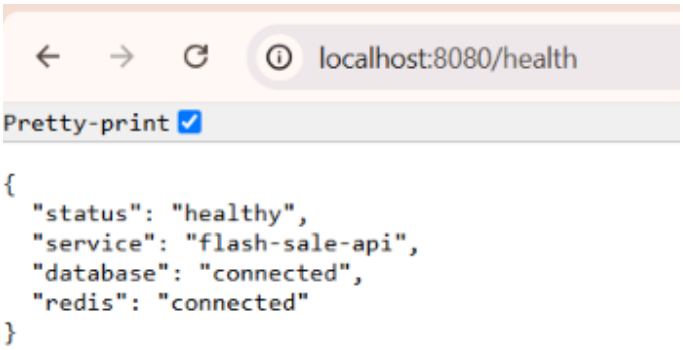


Figure 4: Running Flash-Sale Application Services.

8. Testing & Results

The system was tested locally to evaluate service connectivity, reservation handling, rate limiting, idempotency, asynchronous event processing, and inventory consistency. The tests were conducted using the local Docker Compose environment.

8.1 Test Summary

Test	Observed result
API health check	API health endpoint reported healthy, with PostgreSQL and Redis connectivity verified.
Rate limiting	The first 10 attempts were accepted; the 11th attempt returned HTTP 429 Too Many Requests.
Reservation batch test	510 requests were processed in approximately 3.3 seconds, returning 490 HTTP 202 (Accepted) and 20 HTTP 409 (Conflict) responses. *(Combined with the 10 units reserved during the preceding rate-limit test in Figure 5, total reservations reached exactly 500/500 units with 0 oversold).*
Inventory verification	The database showed 500 units reserved, zero sold, and zero available in the reported batch-test snapshot.
Idempotency	Repeating a request with the same idempotency key returned the existing reservation rather than creating a duplicate.
Reservation expiry	A later database snapshot showed 522 expired reservations, zero reserved units, and all 500 units available again.

8.2 Interpretation of Results

The local tests demonstrate that the implemented system supports reservation requests, rate limiting, idempotent retries, background event processing, and inventory-state verification. The recorded batch-test inventory remained within the configured limit of 500 smartphones.

HTTP 202 indicates that a request was accepted for processing; it does not confirm a completed purchase or successful payment. HTTP 409 indicates a conflict and should be interpreted according to the API's reservation logic.

The reservation-expiry result represents a later test state and should not be treated as the inventory state during the earlier batch test.

8.3 Performance Limitation

The batch test processed 510 requests in approximately 3.3 seconds in a local environment. This result is not evidence that the application can support 100,000 concurrent customers. A dedicated load test is required to measure throughput, latency percentiles, error rates, database contention, and resource utilization under the target workload.



Figure 5: Redis Rate Limiting Rejecting the 11th Reservation Attempt

NOTE:
Attempts 1–10: HTTP 202 Accepted — reservation requests accepted for processing.

Attempt 11: HTTP 429 Too Many Requests — rate limit enforced.

Retry interval: 55 seconds.

```
PS D:\goML-Flash-sale> docker compose exec postgres psql -U flashsale_user -d flashsale -c "SELI
ld_quantity AS available FROM flash_sales; SELECT status, COUNT(*) AS reservation_count FROM re
             code             | total_quantity | reserved_quantity | sold_quantity | available
------------------------------+----------------+-------------------+---------------+-----------
  smartphone - flash - sale   |            500 |                 0 |             0 |       500
(1 row)

 status  | reservation_count
---------+-------------------
 EXPIRED |               515
(1 row)

PS D:\goML-Flash-sale>
```

Figure 6: PostgreSQL Inventory and Reservation State After Testing.

NOTE: The database reports 500 total units, 0 reserved, 0 sold, and 500 available. It also records 515 expired reservations. The inventory values remain within the configured stock limit.

```
PS D:\goML-Flash-sale> docker compose logs --tail=30 worker
worker-1  | Outbox event published {
worker-1  |   eventId: '59d6ea23-88bd-4e50-8877-9ae1d426c341',
worker-1  |   eventType: 'reservation.expired'
worker-1  | }
worker-1  | Event processed {
worker-1  |   eventId: 'f19f72d9-24cc-45ca-ba9d-17f4180033b4',
worker-1  |   eventType: 'reservation.expired',
worker-1  |   reservationId: '59d6ea23-88bd-4e50-8877-9ae1d426c341',
worker-1  |   offset: '368'
worker-1  | }
worker-1  | Outbox event published {
worker-1  |   eventId: '6b2b05d0-d4c8-4757-88b1-a7acdb435d94',
worker-1  |   eventType: 'reservation.expired'
worker-1  | }
worker-1  | Outbox event published {
worker-1  |   eventId: '5b136ecc-d495-4063-bae6-28933334059c',
worker-1  |   eventType: 'reservation.expired'
worker-1  | }
worker-1  | Event processed {
worker-1  |   eventId: '9c818cef-7fb7-43df-baa5-5a36ff6a3e7c',
worker-1  |   eventType: 'reservation.expired',
worker-1  |   reservationId: '6b2b05d0-d4c8-4757-88b1-a7acdb435d94',
worker-1  |   offset: '335'
worker-1  | }
worker-1  | Event processed {
worker-1  |   eventId: '0ec0d708-4c5b-4ab5-8e39-93ee835a3bda',
worker-1  |   eventType: 'reservation.expired',
worker-1  |   reservationId: '5b136ecc-d495-4063-bae6-28933334059c',
worker-1  |   offset: '369'
worker-1  | }
PS D:\goML-Flash-sale>
```

Figure 7 — Background Event Processing

NOTE: The background worker publishes transactional outbox events and processes reservation-expiration events through the event-processing pipeline, demonstrating asynchronous processing.

```
PS D:\goML-Flash-sale> docker compose ps
NAME                  IMAGE                                                              COMMAND                  SERVICE    CREATED             STATUS                    PORTS
flash-sale-api-1      sha256:bf4379245dd54e7bc0e71b0d96e0874475fe603d4c6c977eb5f26f20e9cc23f6   "docker-entrypoint.s…"   api        2 hours ago         Up 2 hours                127.0.0.1:3000->3
000/tcp
flash-sale-frontend-1   flash-sale-frontend                                              "/docker-entrypoint.…"   frontend   About an hour ago   Up About an hour          80/tcp
flash-sale-kafka      apache/kafka:4.1.2                                                 "/__cacert_entrypoin…"   kafka      4 hours ago         Up 4 hours (healthy)      127.0.0.1:9092->2
9092/tcp
flash-sale-nginx-1    nginx:alpine                                                       "/docker-entrypoint.…"   nginx      About an hour ago   Up About an hour          0.0.0.0:8080->80/
tcp, [::]:8080->80/tcp
flash-sale-postgres   postgres:17-alpine                                                 "docker-entrypoint.s…"   postgres   4 hours ago         Up 4 hours (healthy)      127.0.0.1:5432->5
432/tcp
flash-sale-redis      redis:7.4-alpine                                                   "docker-entrypoint.s…"   redis      4 hours ago         Up 4 hours (healthy)      127.0.0.1:6379->6
379/tcp
flash-sale-worker-1   flash-sale-worker                                                  "docker-entrypoint.s…"   worker     2 hours ago         Up 2 hours
PS D:\goML-Flash-sale>
```

Figure 8: Docker Compose Service Status

```
PS D:\goML-Flash-sale> $base = "http://localhost:8080/api/reservations"
>> $key = "idempotency-proof-20261028-01"
>> $body = @{
>>     customerId = "idempotency-proof-test"
>>     quantity = 1
>> } | ConvertTo-Json
>>
>> $first = Invoke-WebRequest -Uri $base -Method POST `
>>     -Headers @{ "Idempotency-Key" = $key } `
>>     -ContentType "application/json" -Body $body
>>
>> "FIRST REQUEST: HTTP $([int]$first.StatusCode)"
>> $first.Content
FIRST REQUEST: HTTP 202
{"message":"Reservation accepted.","reservationId":"0c61771d-dea7-4730-9c00-4e4b0e1fd47c","flash
0-28T16:58:17.864Z","replayed":false}
PS D:\goML-Flash-sale> $second = Invoke-WebRequest -Uri $base -Method POST `
>>     -Headers @{ "Idempotency-Key" = $key } `
>>     -ContentType "application/json" -Body $body
>>
>> "SECOND REQUEST: HTTP $([int]$second.StatusCode)"
>> $second.Content
SECOND REQUEST: HTTP 200
{"reservationId":"0c61771d-dea7-4730-9c00-4e4b0e1fd47c","flashSaleId":"b470de08-c44f-483e-938a-5
e}
PS D:\goML-Flash-sale>
```

Figure 9: Idempotency Preventing Duplicate Reservations

NOTE: The initial reservation request returned HTTP 202. Repeating the same request with the same idempotency key returned HTTP 200 with the same reservation ID and replayed: true, confirming that the retry did not create a duplicate reservation.

```
PS D:\goML-Flash-sale> docker compose exec postgres psql -U flashsale_user -d flashsale -
 status ORDER BY status; SELECT total_quantity, reserved_quantity, sold_quantity, (reserv
 status  | reservations | units
---------+--------------+-------
 EXPIRED |          516 |   516
 PENDING |            6 |     6
(2 rows)

 total_quantity | reserved_quantity | sold_quantity | inventory_within_limit
----------------+-------------------+---------------+------------------------
            500 |                 6 |             0 | t
(1 row)

PS D:\goML-Flash-sale>
```

Figure 10: PostgreSQL Inventory Constraint Verification

PostgreSQL confirmed that six reservations were pending, accounting for six units. With 500 total units and zero sold, 494 units remained available. The inventory constraint check returned true, confirming that reserved and sold quantities did not exceed total stock at the time of verification.

```
PS D:\goML-Flash-sale> docker compose exec postgres psql -U flashsale
l_quantity, reserved_quantity, sold_quantity, total_quantity - reserv
 status  | reservations
---------+--------------
 EXPIRED |          522
(1 row)

 total_quantity | reserved_quantity | sold_quantity | available
----------------+-------------------+---------------+-----------
            500 |                 0 |             0 |       500
(1 row)

PS D:\goML-Flash-sale>
```

Figure 11: Reservation Expiry and Inventory Release

After the reservation expiry process completed, all 522 reservations were marked as expired. PostgreSQL showed zero reserved units and all 500 units available again, demonstrating that expired reservations release inventory back to the flash sale.

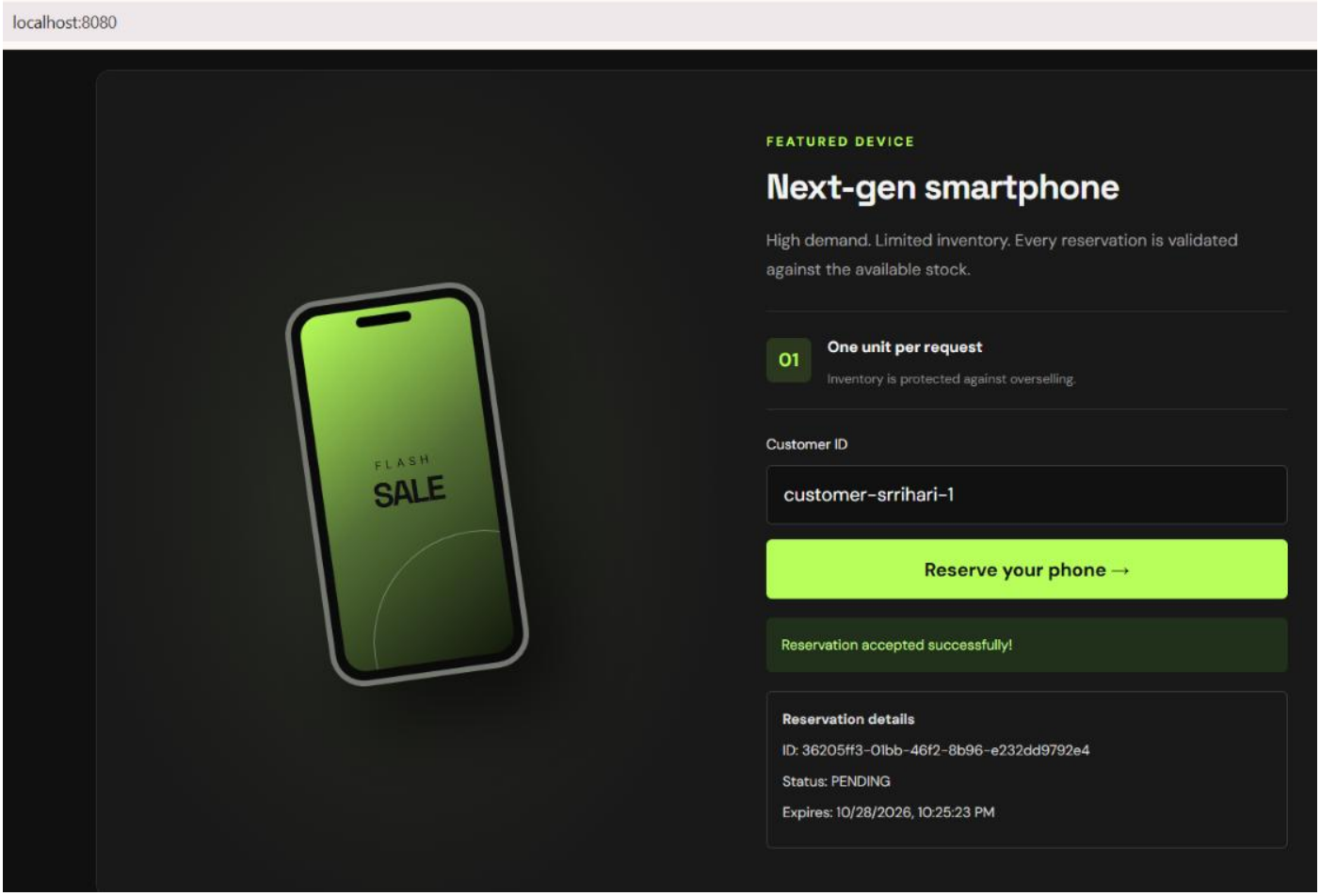


Figure 12: Flash-Sale Application Interface and Successful Reservation Confirmation

9. Scalability, Reliability & Security

Scalability: The application separates the frontend, API, database, cache, and background worker into services. Docker Compose is used for local execution. Horizontal scaling and load balancing are possible improvements, but performance at the target scale has not been validated.

Reliability: The implementation uses PostgreSQL transaction-based inventory handling, Redis rate limiting, idempotency checks, and Kafka-based asynchronous event processing with a transactional outbox. Failure recovery, duplicate-event handling, and end-to-end event delivery should be validated through dedicated tests before production deployment.

Security: Input validation, parameterized database queries, idempotency checks, and rate limiting are described as implemented measures. HTTPS, secure secret management, authentication, authorization, and least-privilege access should be validated or added before production deployment.

Production readiness: The current implementation is a local prototype. Distributed load testing, monitoring, failure-recovery testing, security validation, and infrastructure-capacity planning remain necessary.

The architecture provides a foundation for handling flash-sale traffic, but production readiness requires load testing, monitoring, failure-recovery testing, and security validation.

10. Limitations & Conclusion

Limitations

- The system has been tested locally; support for 100,000 concurrent customers has not yet been validated.
- Production deployment would require distributed load testing, monitoring, automated recovery, and infrastructure scaling.
- Reservation acceptance does not necessarily indicate completed payment or purchase.

Conclusion

The Flash-Sale System addresses the challenge of allocating 500 smartphones under high demand using PostgreSQL transactions, Redis rate limiting, idempotency, and Kafka-based asynchronous processing. The local tests demonstrated reservation handling and inventory protection under the tested workload. Further load and failure-recovery testing is required before claiming production readiness at 100,000 concurrent customers.